ARCHITECTURAL FRAMEWORK PLAYBOOK

Spec-Driven Development (SDD) with AWS Kiro

In modern production ecosystems, moving away from ad-hoc prompting ("vibe coding") is critical to preventing architectural decay and unexpected code drift. **Spec-Driven Development (SDD)** enforces a strict, multi-stage gate: **Requirements → Design → Tasks → Implementation**. AWS Kiro structures this progression transparently within your file system under the `.kiro/specs/` directory, demanding explicit sign-off from engineers before executing changes.

1. Blueprint Workflow: Adding a Feature to a Blog

To demonstrate the structure, consider adding a **`Last Updated`** timestamp to a markdown-driven blog layout. Below is how AWS Kiro maps and executes this lifecycle across three core markdown specifications.

Phase 1: Requirements Gathering (`requirements.md`)

When initialized, Kiro maps user intent into standardized engineering goals using **User Stories** and **EARS (Easy Approach to Requirements Syntax)** notation.

```
# Requirements: Blog Last Updated Timestamp

## Requirements & Behavioral Logic
### Requirement 1: Conditional Display
**User Story:** As a reader, I want to see when a post was last updated so that I know the i

#### Acceptance Criteria
- WHEN a blog post has an 'updated_at' frontmatter field, THEN the system SHALL display "Las
- IF a blog post only has a 'date' field, THEN the system SHALL NOT display the "Last Update
- WHEN the 'updated_at' date is identical to the 'date' field, THEN the system SHALL suppres
```

> **Kiro Quality Gate 1**
>
> The system pauses code generation entirely. The operator must verify edge cases (e.g., date parity) within the workspace interface before moving to design synthesis.

Phase 2: Technical Design (`design.md`)

Upon approval, Kiro parses the workspace schema and defines precise layout modifications, constraints, and dependencies.

```
# Technical Design: Blog Last Updated Timestamp

## Data Schema & Structural Changes
1. **Schema Update:** Update Markdown schema validation to optionally accept an 'updated_at'
2. **Component Layer (src/_includes/layouts/post.njk):**
   - Inject a structural evaluation utility checking if 'updated_at' > 'date'.
   - Append appropriate utility classes for styling.

## Error Boundary Handling
- If 'updated_at' is malformed, log a build warning and default safely back to initial publi
```

Phase 3: Task Breakdown & Parallel Execution (`tasks.md`)

Kiro translates approved design logic into stateful task arrays, optimizing the implementation pipeline into concurrent waves via dependency graphs.

```
# Task List: Blog Last Updated Timestamp

- [ ] **Task 1: Date formatting utility**
  - Create date filter to handle string comparisons.
  - *Definition of Done:* Unit test passes with matching and non-matching instances.
- [ ] **Task 2: Update post layout template**
  - Modify src/_includes/layouts/post.njk to support the conditional DOM block.
```

2. Hands-On Activity: Build an API Rate Limiter

This hands-on exercise guides you through implementing an authenticated **Rate Limiter Middleware** for an Express/Node.js API using Kiro's strict SDD gates.

1

Initialize the Spec Session

Open the Kiro UI pane in your IDE or enter the `/spec` command in the terminal prompt. Issue the following high-level instruction:

"Create a rate limiter middleware for our Express API that restricts users to a maximum number of requests within a sliding window. If they exceed it, return an HTTP 429 status code."

2

Refine and Lock Requirements

Navigate to `.kiro/specs/rate-limiter/requirements.md`. Review and expand the criteria to include explicit response header fields: Ensure the rules verify that the client is identified by IP address or **`Authorization`** tokens, and return headers like **`X-RateLimit-Limit`** and **`X-RateLimit-Remaining`**. Once validated, input: **`"Requirements approved, generate technical design."`**

3

Audit Architecture and Steering Controls

Open the resulting `design.md`. Kiro maps the stack relative to your local steering profiles (e.g., configuring an memory cache or Redis). If Kiro defaults to a basic fixed-window design, explicitly instruct: *"Update the design to leverage a sliding window log structure using Redis sorted sets to preserve transaction accuracy."*

4

Execute and Monitor Autonomous Compilations

Confirm the final task breakdowns inside `tasks.md` and select **"Run All Tasks"**. Kiro spins up concurrent task-specific instances to write matching unit tests and middleware logic simultaneously. Watch the states transition smoothly from **`[in-progress]`** to checked-off **`[x]`** statuses automatically as validation guards clear.

> **Reference & Visual Media Resource**
>
> For a comprehensive breakdown of the Kiro interface mechanics, project environment structures, and a deep comparison between fast "Vibe Coding" vs. "Spec Mode", consult the operational video: **"KIRO For Dummies: The Complete Beginner's Guide"** (Available via online media search index).